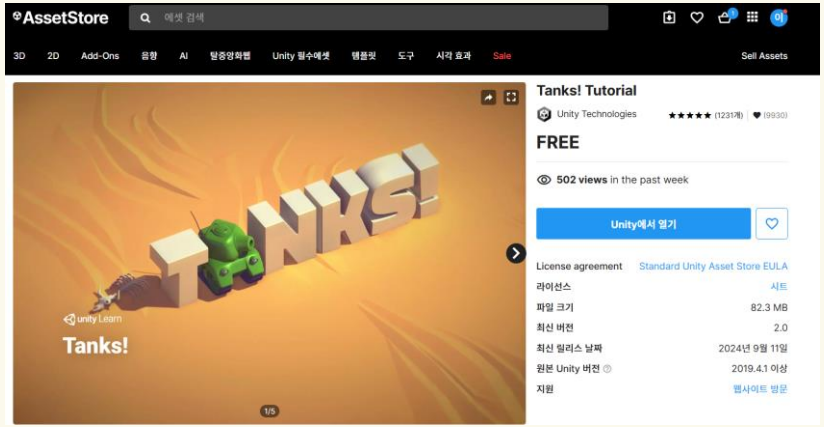
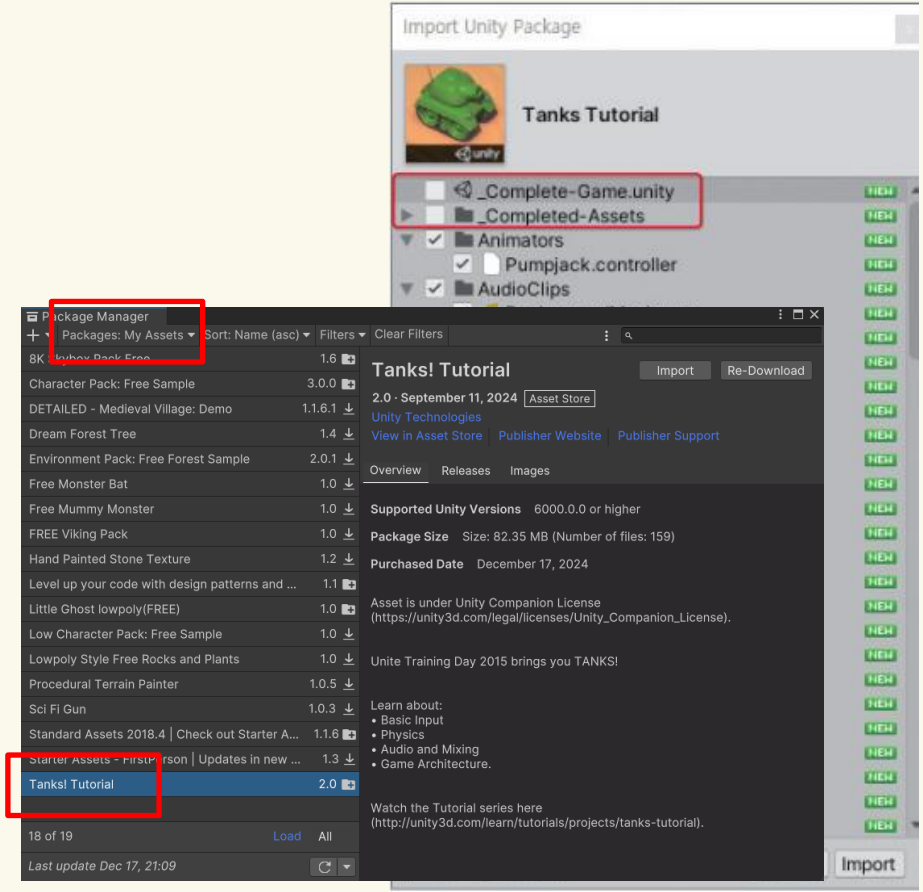
유니티 2인용게임 & 이팩트

CODING SOFTWARE CREATIVE EDUCATION CENTER

2인용 게임 만들기

- 에셋 가져오기
  - AssetStore : Tanks! Tutorial

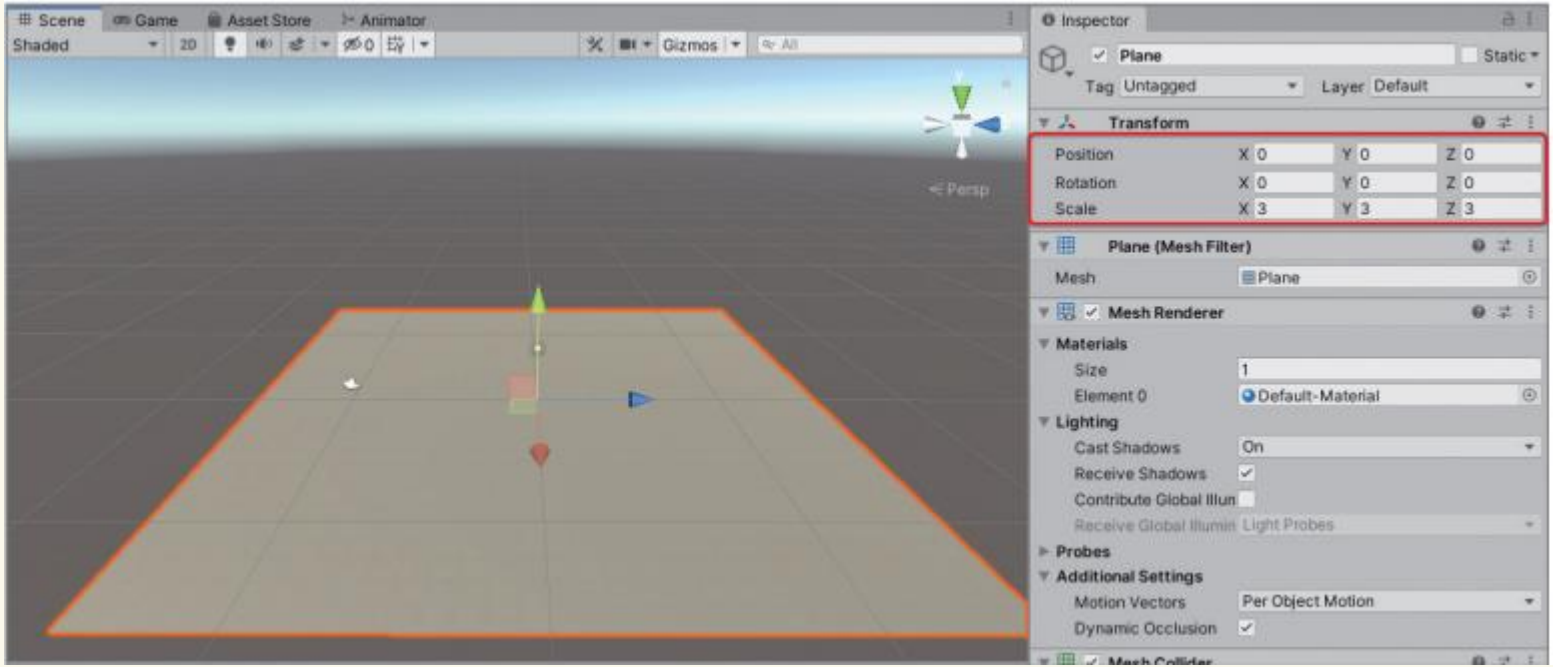




[ej070829@gmail.com, !JejuCoding6560]

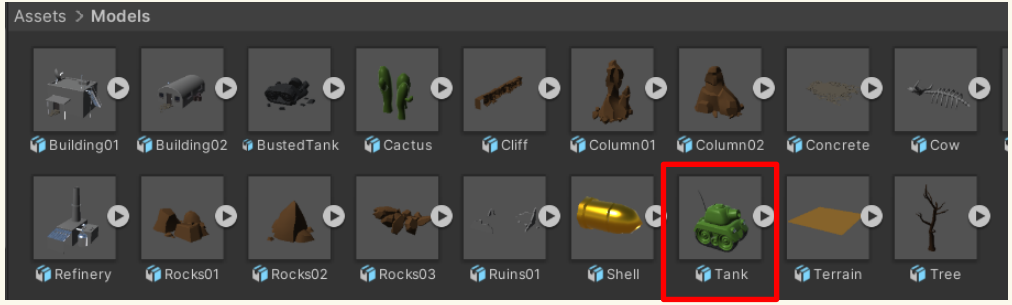
2인용 게임 만들기

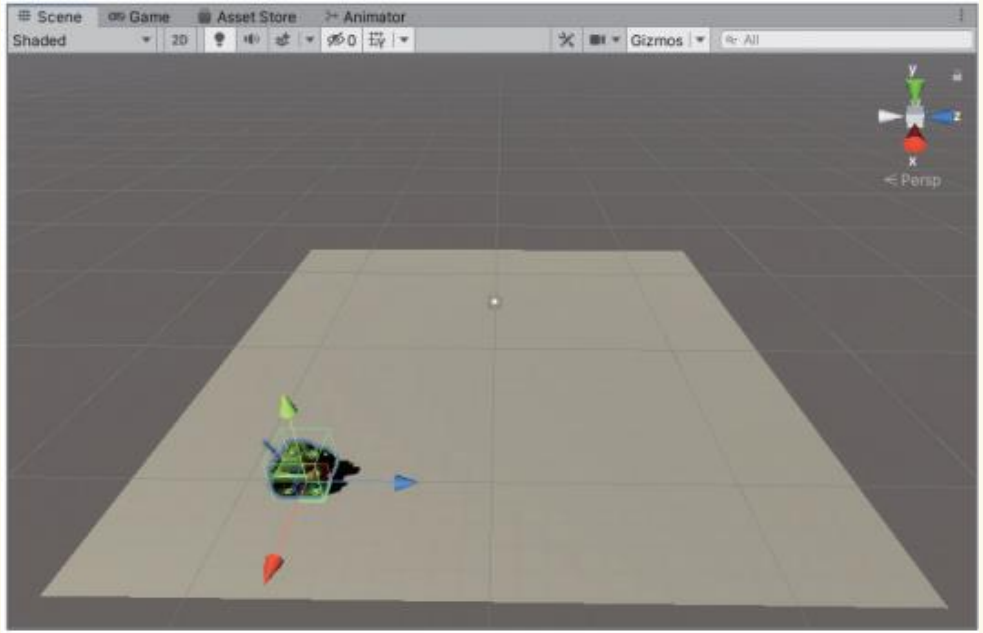
- Plane 추가
  - Scale 조정 및 Main Camera 확인

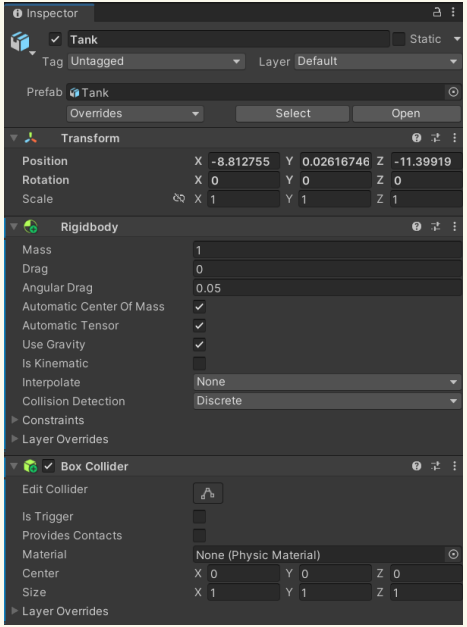


2인용 게임 만들기

- Models 폴더 : Tank 가져오기
  - Rigidbody 컴포넌트 지정
  - Box Collider 지정

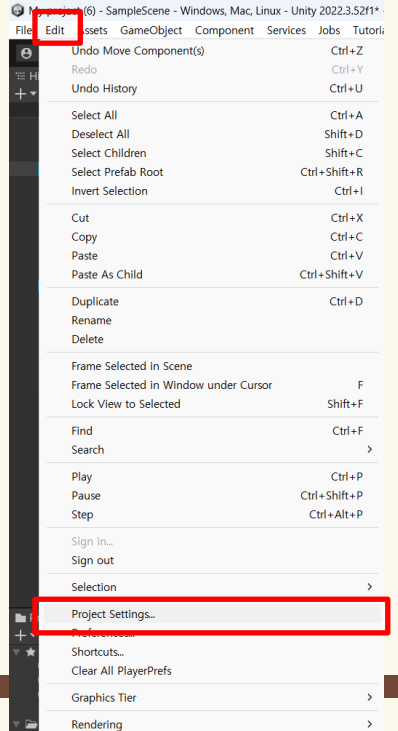


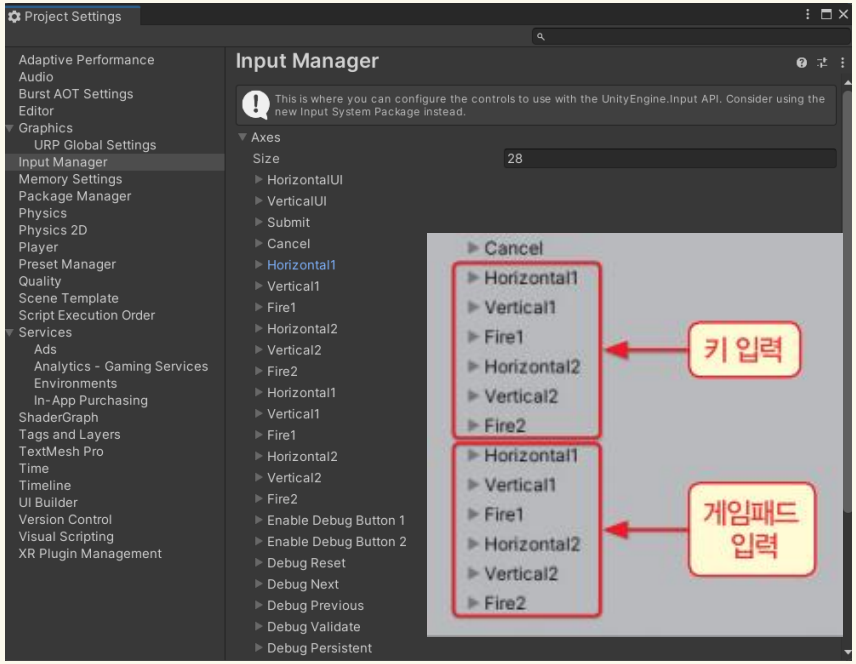




2인용 게임 만들기

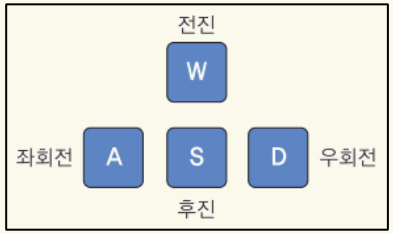
- 입력 관리자 설정
  - Input.GetAxis( ) 메서드 사용 : 수평축(Horizontal)과 수직축(Vertical)을 입력으로 사용
  - [Edit]-[Project Settings] 클릭 -> [Project Settings] 창 : Horizontal1, Vertical1을 사용



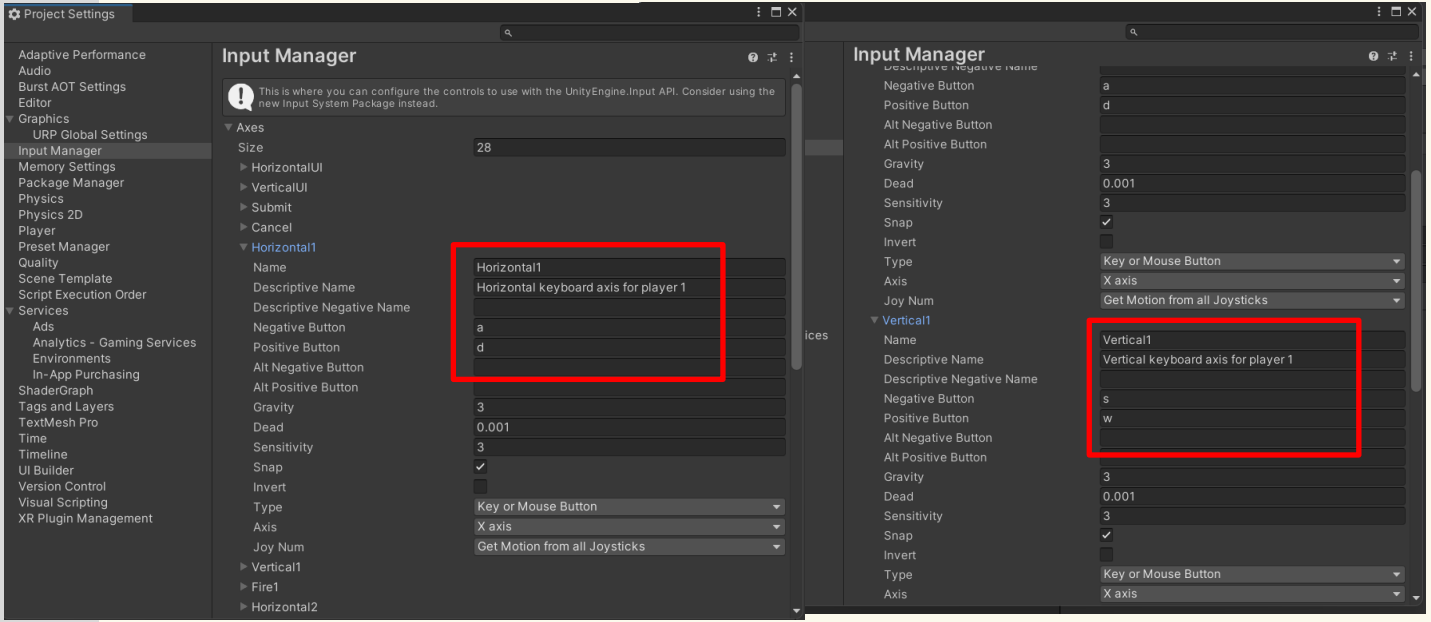


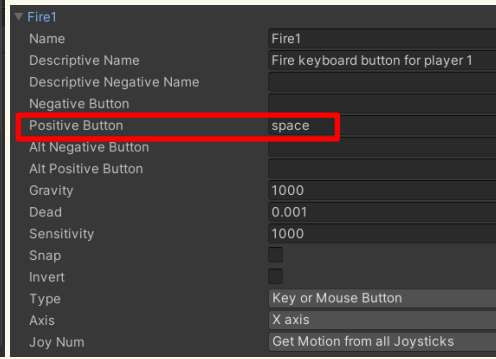
2인용 게임 만들기

- 입력 관리자 설정
  - Input.GetAxis( ) 메서드 사용 : 수평축(Horizontal)과 수직축(Vertical)을 입력으로 사용
  - [Edit]-[Project Settings] 클릭 -> [Project Settings] 창 : Horizontal1, Vertical1을 사용



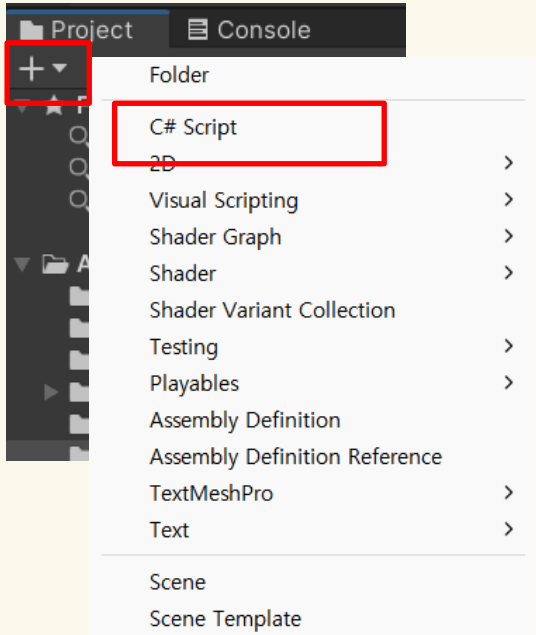
버튼 : space





2인용 게임 만들기

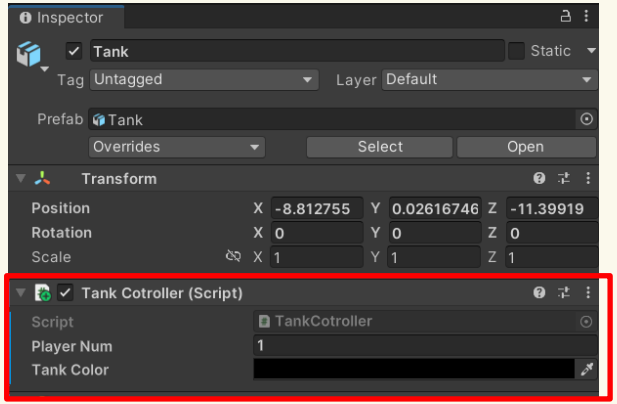
- C# Script : TankCotroller




```
public class TankCotroller : MonoBehaviour
{
    float tkMvSpeed = 10.0f;        // 탱크의 이동 속도
    float tkRotSpeed = 150.0f;      // 탱크의 회전 속도

    void Update()
    {
        float mv = Input.GetAxis ("Vertical1") * tkMvSpeed * Time.deltaTime;
        float rot = Input.GetAxis ("Horizontal1") * tkRotSpeed * Time.deltaTime;

        transform.Translate (0 , 0 , mv);
        transform.Rotate (0 , rot , 0);
    }
}
```

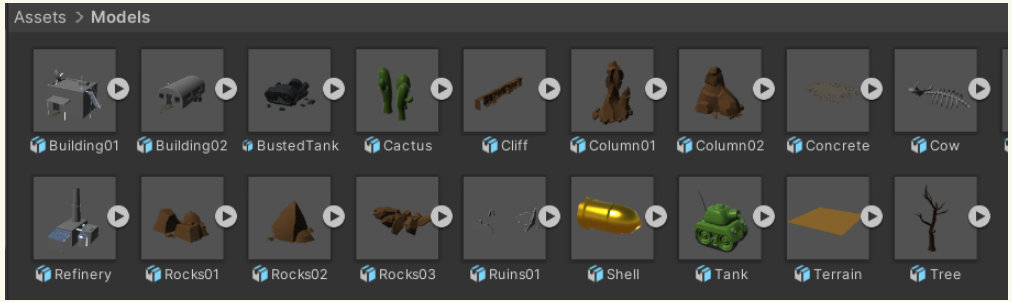
Vertical1로 전후진

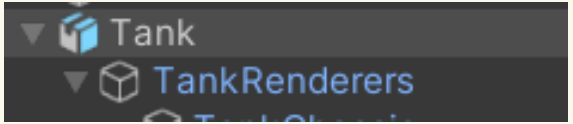
Horizontal1로 회전

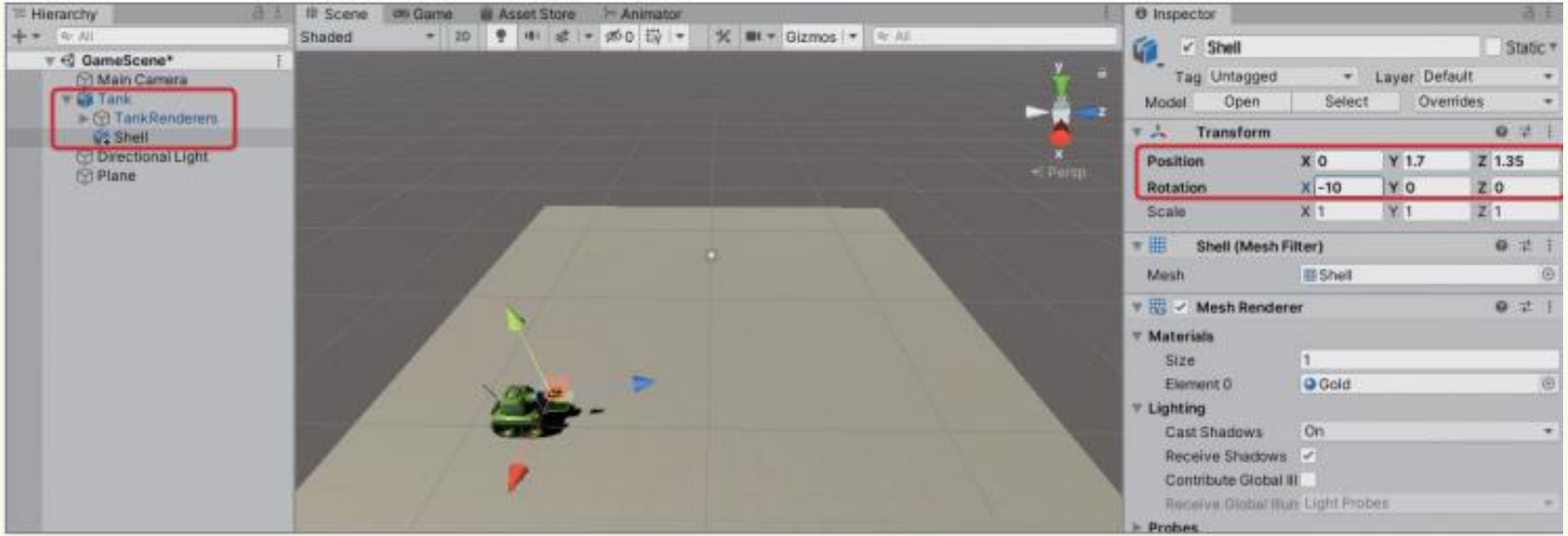


2인용 게임 만들기

- Shell 오브젝트 : 프리팹 만들기
  - Models – Shell

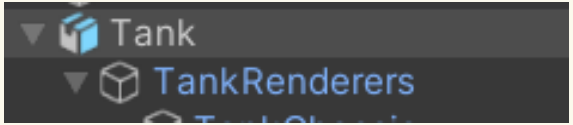


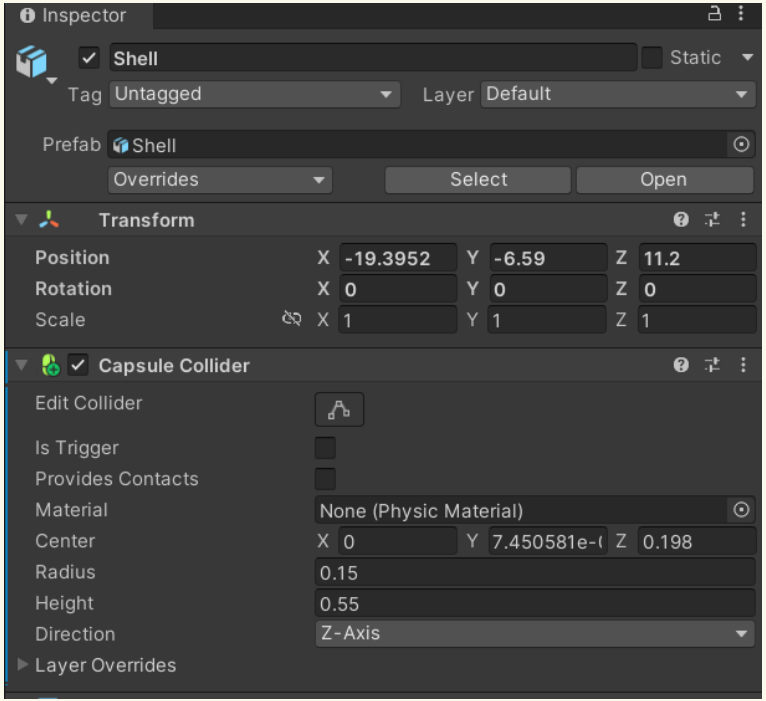


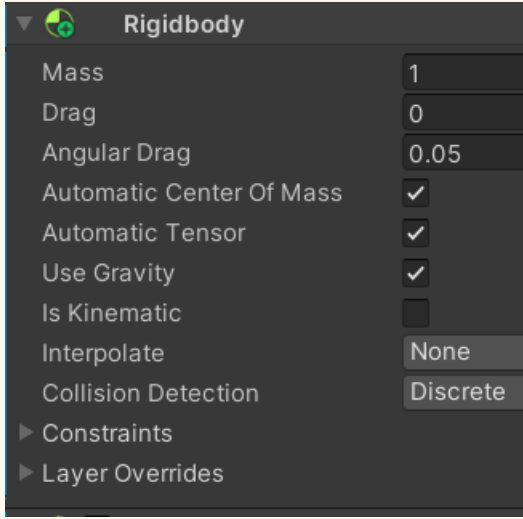


2인용 게임 만들기

- Tank – TankRenderers 하위 폴더 : Shell 오브젝터



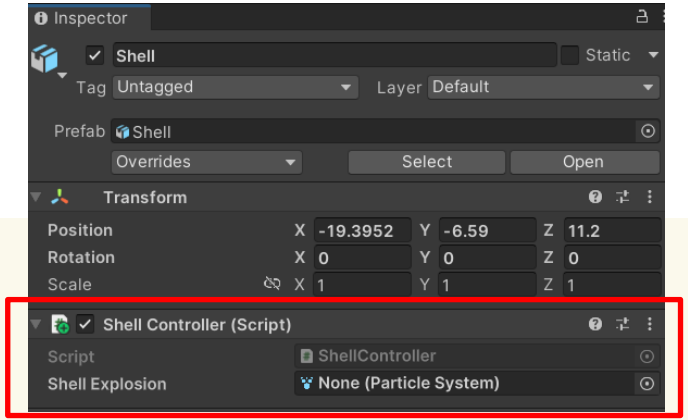




2인용 게임 만들기

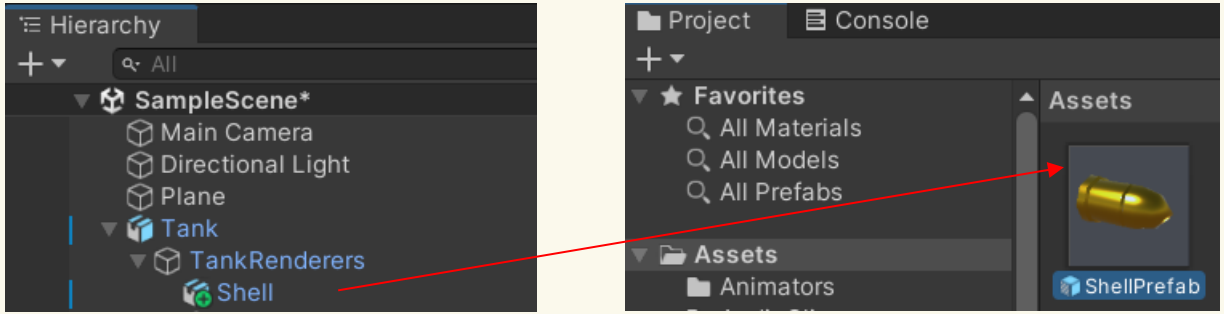
- 포탄 쏘기
  - Shell 오브젝트 스크립트 : ShellController

```
public class ShellController : MonoBehaviour
{
    public ParticleSystem shellExplosion;

    private void OnCollisionEnter (Collision other)
    {
        ParticleSystem fire = Instantiate (shellExplosion , transform.position , Quaternion.identity);
        fire.Play ();

        Destroy (gameObject);
        Destroy (fire.gameObject , 2.0f);
    }
}
```

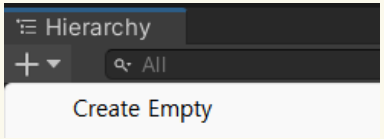


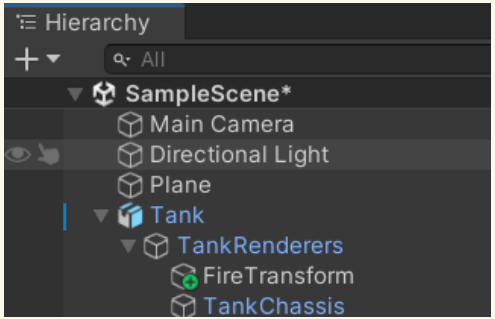
2인용 게임 만들기

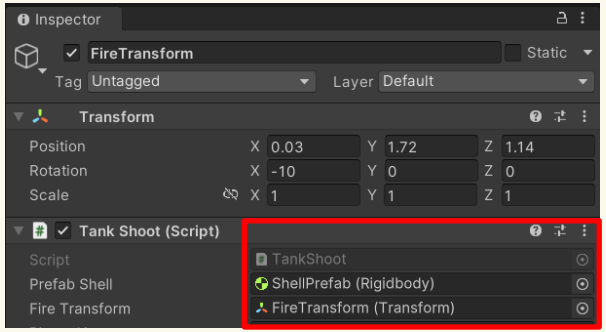
- Shell 오브젝트를 프로젝트창으로 드래그하여 **ShellPrefab 생성**



- **빈 오브젝트 만들기 : FireTransform + 스크립트 : TankShoot + Shellperfab**





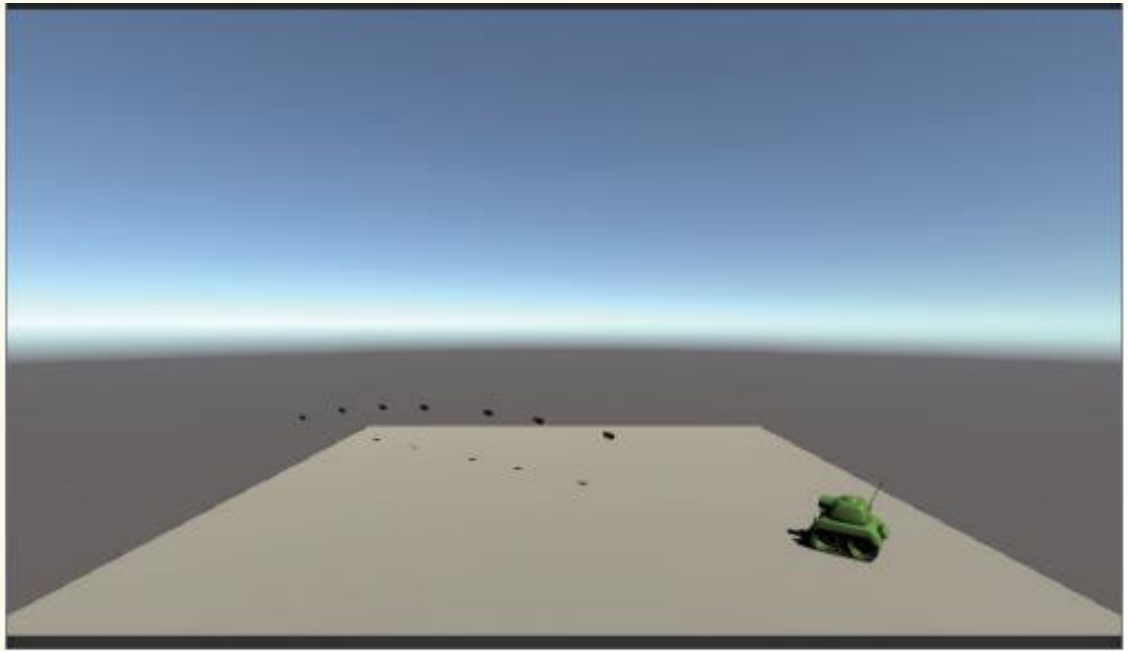


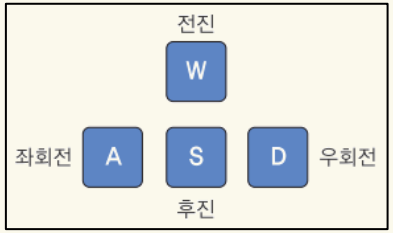
2인용 게임 만들기

- Shell 프리팹 발사
  - FireTransform 오브젝트 스크립트 : TankShoot

```
public class TankShoot : MonoBehaviour
{
    public Rigidbody prefabShell;
    public Transform fireTransform;

    void Update()
    {
        if (Input.GetButtonDown ("Fire1")) {
            Fire ();
        }
    }

    void Fire ()
    {
        Rigidbody shellInstance =
                Instantiate (prefabShell , fireTransform.position , fireTransform.rotation) as Rigidbody;

        // Set the shell's velocity to the launch force in the fire position's forward direction.
        shellInstance.velocity = 20.0f * fireTransform.forward;
    }
}
```

2인용 게임 만들기

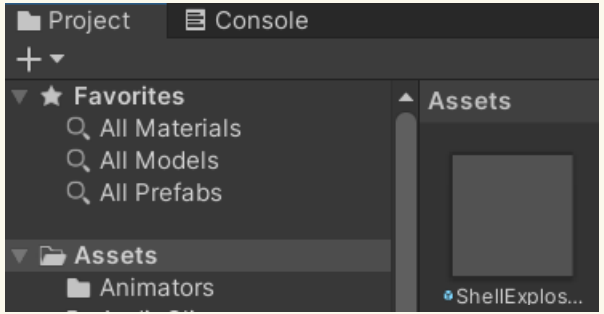




버튼 : space

이펙트 사용하기

- Shell 발사 후 폭파하는 이팩트 적용하기
  - 게임 이펙트 : 게임 효과를 의미
  - [TanksAssets]-[Prefabs]를 선택하고 ShellExplosion 프리팹을 Assets로 복사

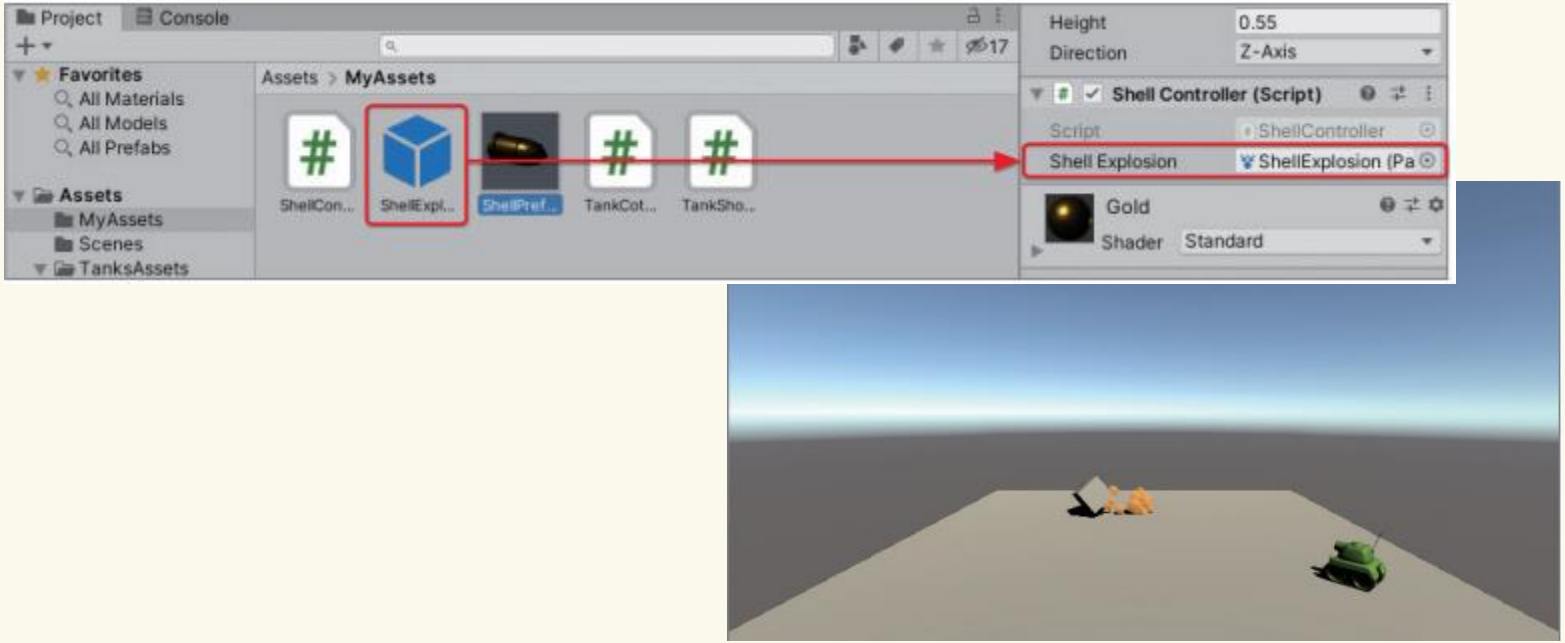



```
public class ShellController : MonoBehaviour
{
    public ParticleSystem shellExplosion;

    private void OnCollisionEnter (Collision other)
    {
        ParticleSystem fire = Instantiate (shellExplosion , transform.position , Quaternion.identity);
        fire.Play ();

        Destroy (gameObject);
        Destroy (fire.gameObject , 2.0f);
    }
}
```

  - ShellController 스크립트에서 충돌시 ShellExplosion 프리팹으로 폭파 이펙트 인스턴스를 만들고 이 인스턴스를 플레이
  - ShellExplosion 프리팹의 정보를 저장하는 변수가 필요
  - 이펙트는 애니메이션처럼 Play( ) 함수를 호출하는 코드를 추가

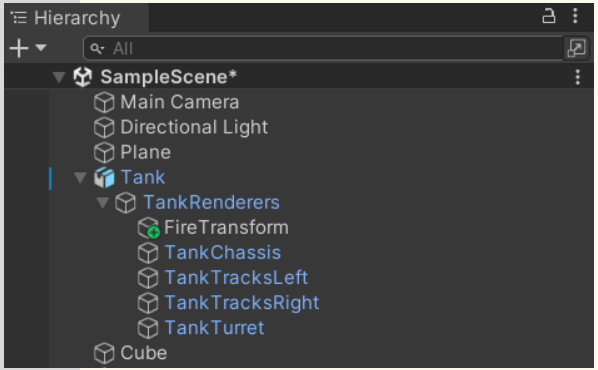
이펙트 사용하기

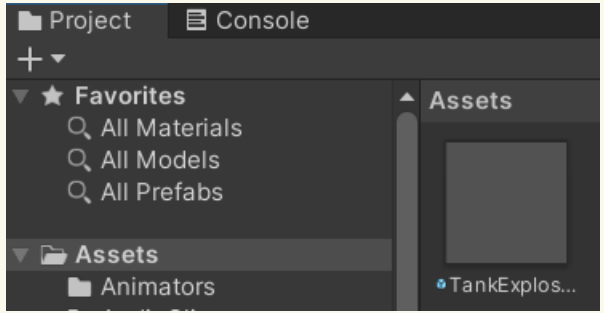
- Shell 발사 후 폭파하는 이팩트 적용하기
  - ShellExplosion 프리팹 지정

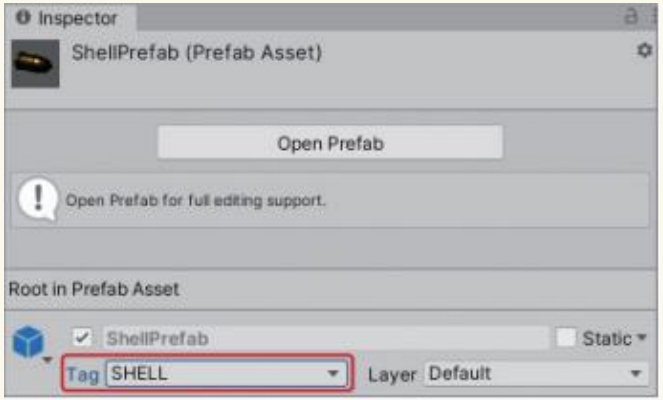


이펙트 사용하기

- 목표물 폭파 이펙트
  - 목표물 : Cube 오브젝트 추가
  - [TanksAssets]-[Prefabs]를 선택하고 Tank Explosion 프리팹을 Assets로 복사
  - 목표물은 shell과 충돌하는 경우에만 폭파 이펙트가 실행되도록 태그처리



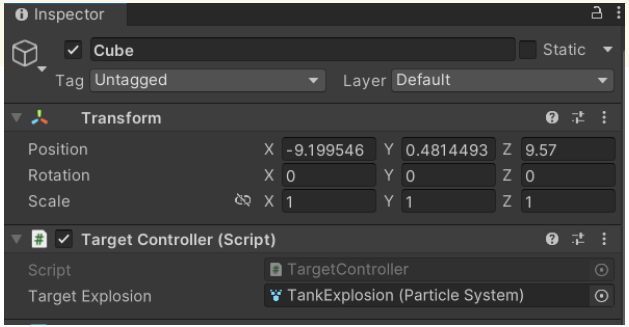


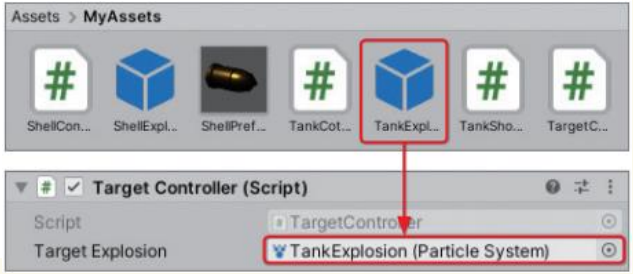


이펙트 사용하기

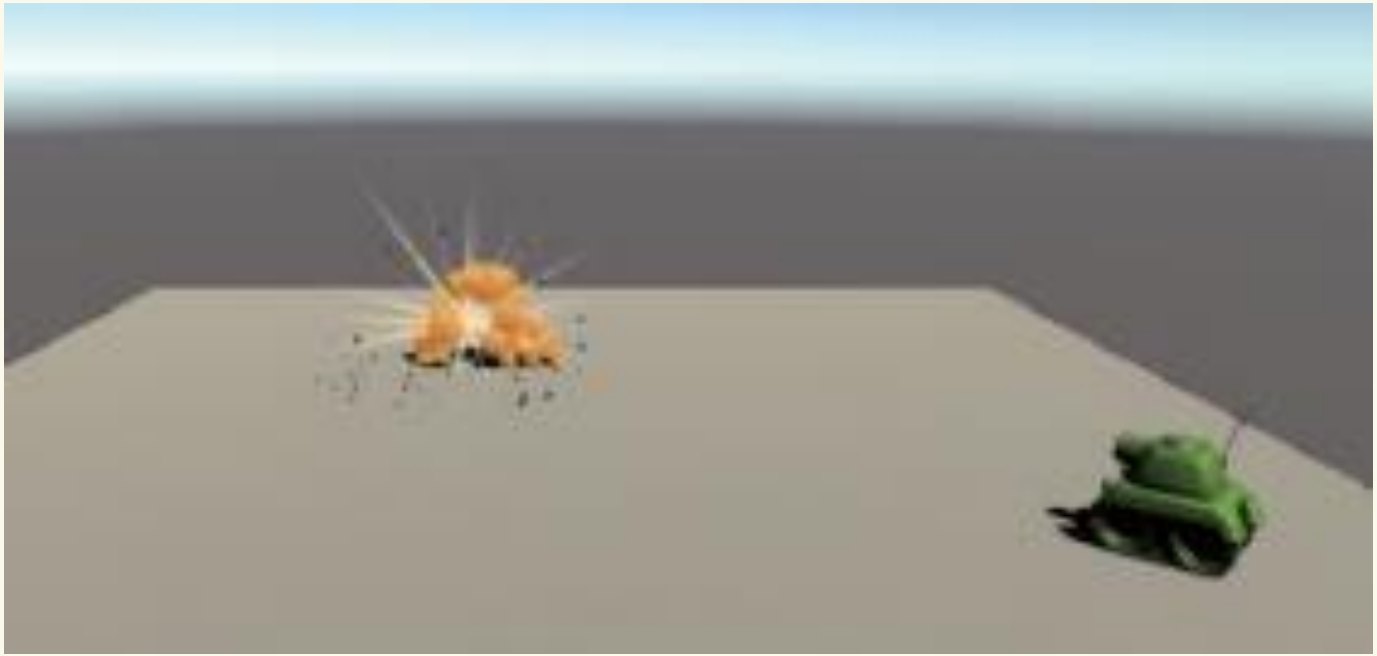
- 목표물 Cube에 폭파 이벤트 처리를 위한 스크립트 : TargetController

```
public class TargetController : MonoBehaviour
{
    public ParticleSystem targetExplosion;

    private void OnCollisionEnter (Collision coll)
    {
        if (coll.collider.tag == "SHELL") {
            ParticleSystem fire = Instantiate (targetExplosion , transform.position , Quaternion.identity);
            fire.Play ();

            Destroy (gameObject);
            Destroy (fire.gameObject , 2.0f);
        }
    }
}
```



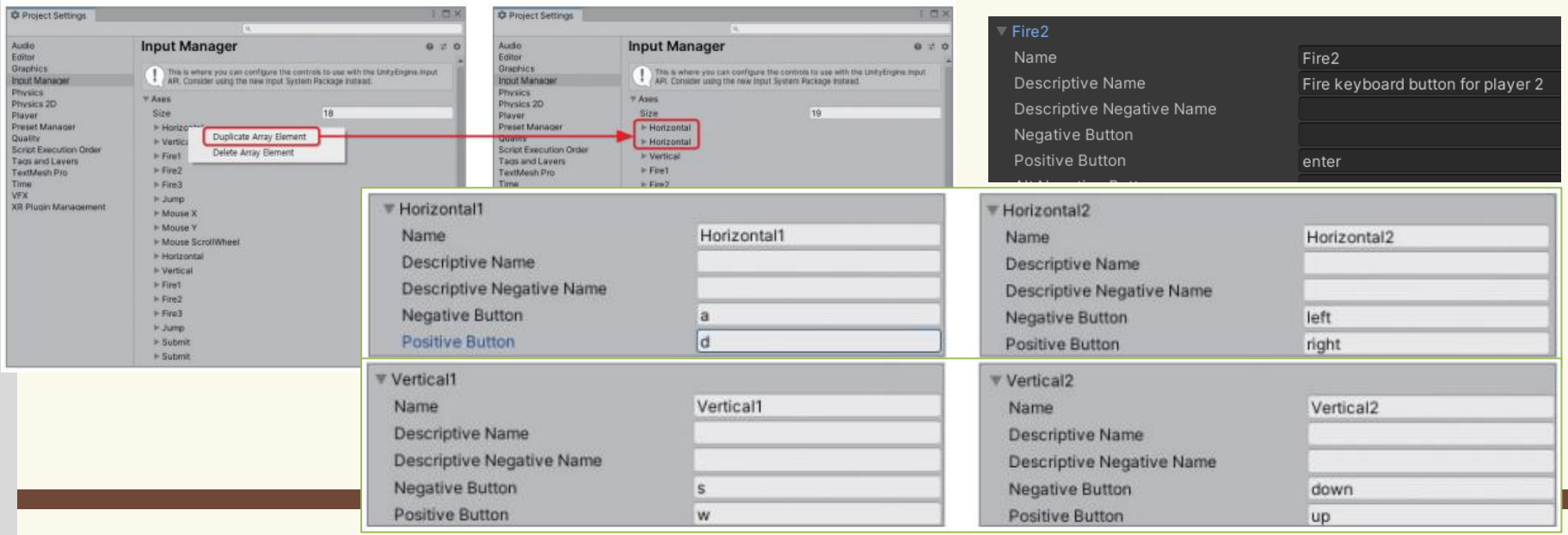


이펙트 사용하기



2인용 게임 만들기

- 입력 관리자 확인하기[[Edit]-[Project Settings] 클릭]
  - 입력 키로 입력 관리자의 1번 설정(Horizontal1, Vertical1, Fire1)을 사용
  - 두 번째 탱크는 입력 키의 2번 설정(Horizontal2, Vertical2, Fire2)을 사용



2인용 게임 만들기

- TankController 스크립트 수정 :

  -> TankController2

  - 플레이어 번호를 사용하여 구분

```
public class TankCotroller : MonoBehaviour
{
    public int playerNum = 1;        // 플레이어 번호
    string mvAxisName;                // 이동 축의 이름 저장
    string rotAxisName;               // 회전 축의 이름 저장
    float tkMvSpeed = 10.0f;        // 탱크의 이동 속도
    float tkRotSpeed = 150.0f;      // 탱크의 회전 속도
    public Color tankColor;

    void Start()
    {
        mvAxisName = "Vertical" + playerNum;
        rotAxisName = "Horizontal" + playerNum;

        // Get all of the renderers of the tank.
        MeshRenderer [ ] renderers = GetComponentsInChildren<MeshRenderer> ();

        // Go through all the renderers...
        for (int i = 0; i < renderers.Length; i++) {
            // ... set their material color to the color specific to this tank.
            renderers [ i ].material.color = tankColor;
        }
    }
    void Update()
    {
        float mv = Input.GetAxis (mvAxisName) * tkMvSpeed * Time.deltaTime;
        float rot = Input.GetAxis (rotAxisName) * tkRotSpeed * Time.deltaTime;

        transform.Translate (0 , 0 , mv);
        transform.Rotate (0 , rot , 0);
    }
}
```

2인용 게임 만들기

- TankShoot 스크립트 수정 :

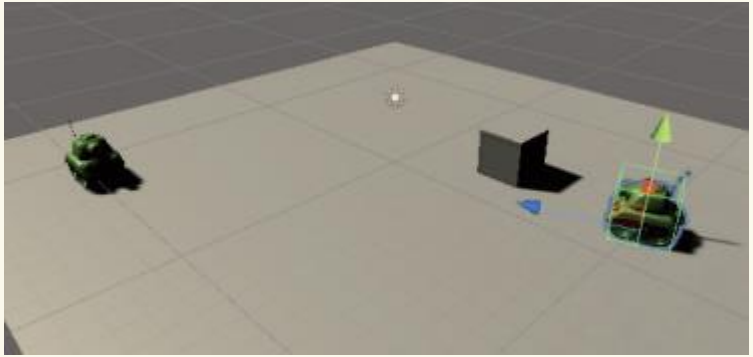
  -> TankShoot2

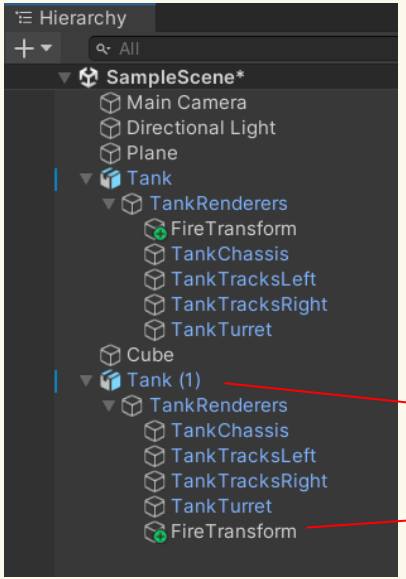
  - 플레이어 번호를 사용하여 구분

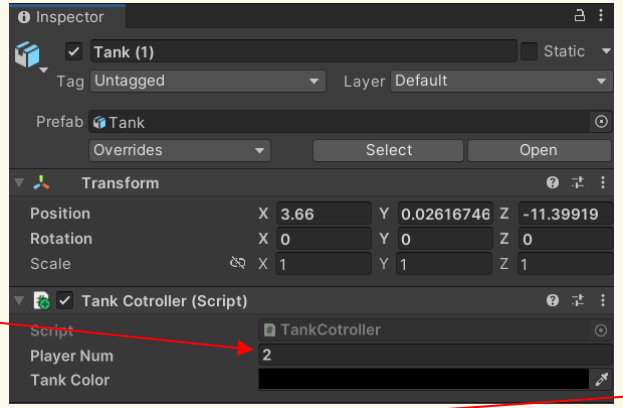
```
public class TankShoot : MonoBehaviour
{
    public Rigidbody prefabShell;
    public Transform fireTransform;
    public int playerNum = 1;        // 플레이어 번호
    string fireName;                    // 발사키 이름 저장

    // Start is called before the first frame update
    void Start ()
    {
        fireName = "Fire" + playerNum;
    }
    void Update()
    {
        if (Input.GetButtonDown (fireName)) {
            Fire ();
        }
    }
    void Fire ()
    {
        Rigidbody shellInstance =
                Instantiate (prefabShell , fireTransform.position , fireTransform.rotation) as Rigidbody;

        // Set the shell's velocity to the launch force in the fire position's forward direction.
        shellInstance.velocity = 20.0f * fireTransform.forward;
    }
}
```

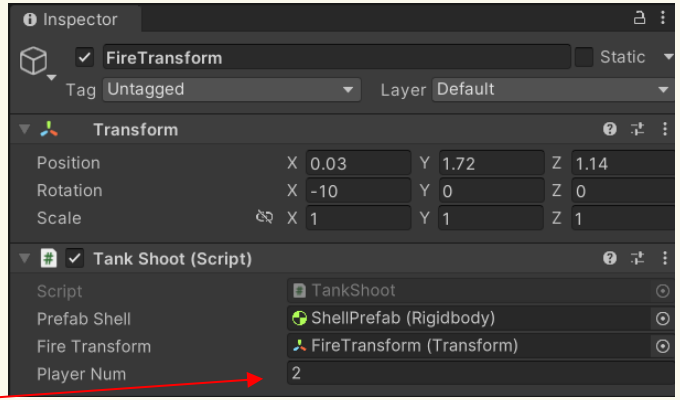
2인용 게임 만들기

- 두 번째 탱크 만들기
  - Tank 오브젝트 복사 -> Tank2로
  - Tank2의 플레이어 번호를 2로 조정

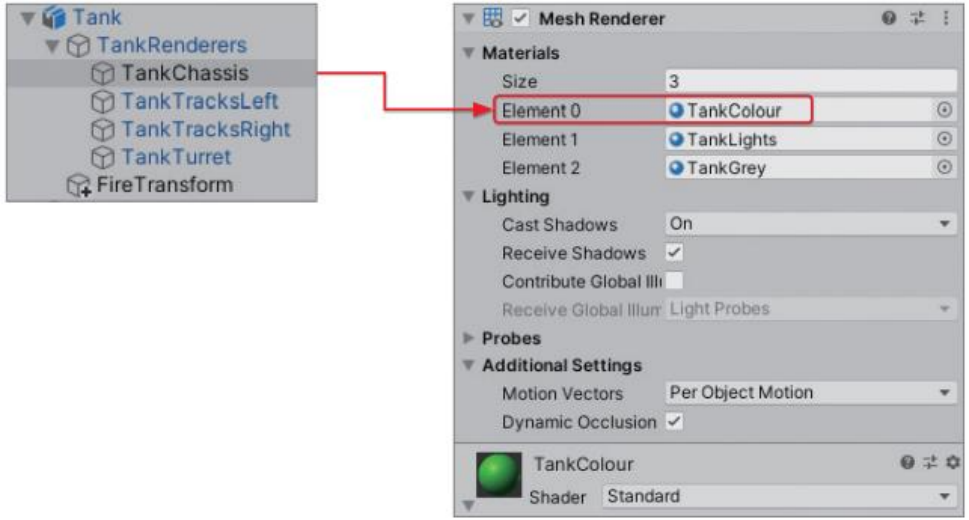






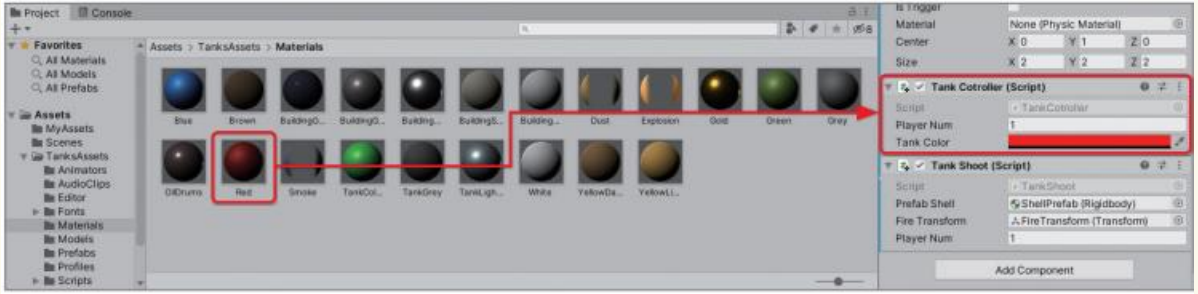


- 탱크의 컬러 조정
  - 탱크를 구성하는 요소인 TankChassis, TankTracksLeft, TankTracksRight, Tank Turret 오브젝트는 각각 Mesh Renderder 컴포넌트를 가지고 있는데 여기서 컬러지정 가능



2인용 게임 만들기

- 탱크의 컬러 조정
  - 유니티 에디터에서 Tank와 Tank2 오브젝트를 각각 선택하여 TankController 스크립트의 전역변수인 Tank Color에 색상을 지정



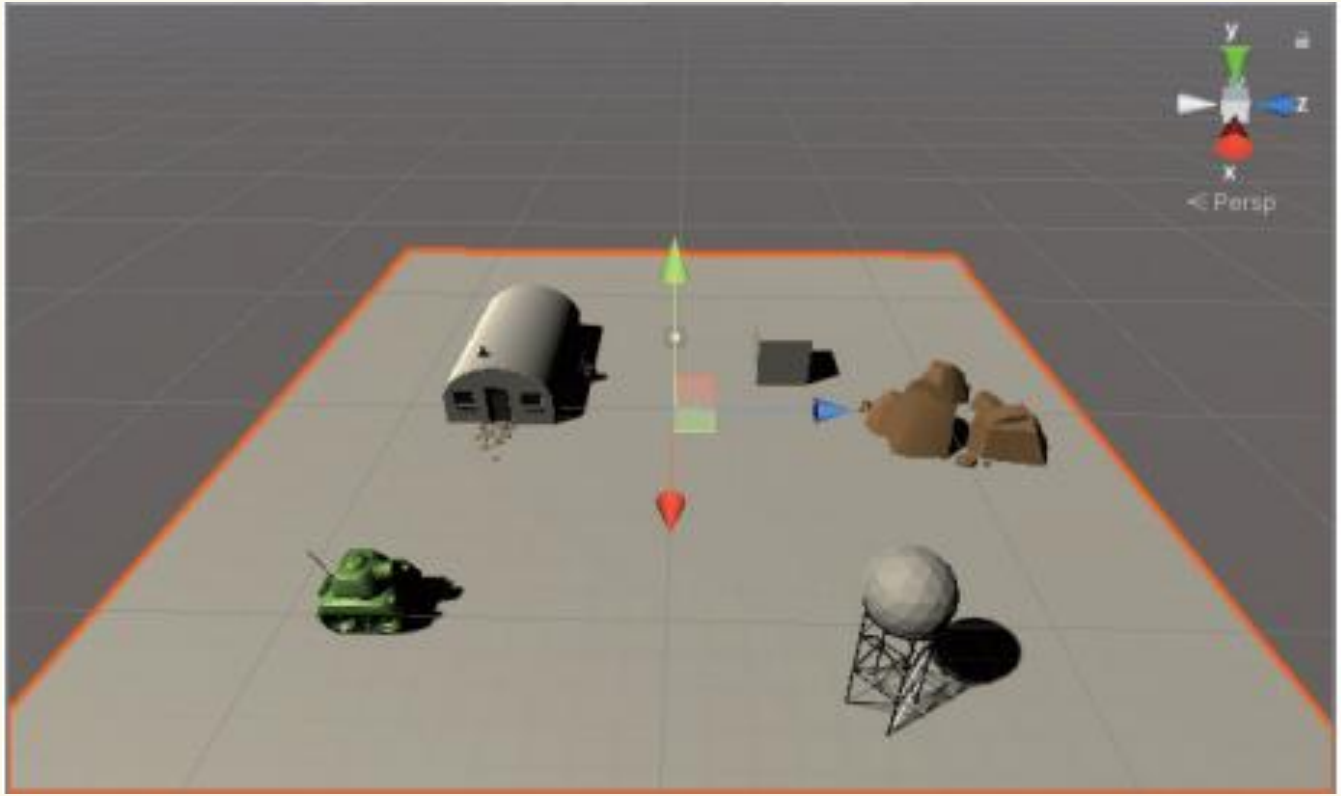
2인용 게임 만들기

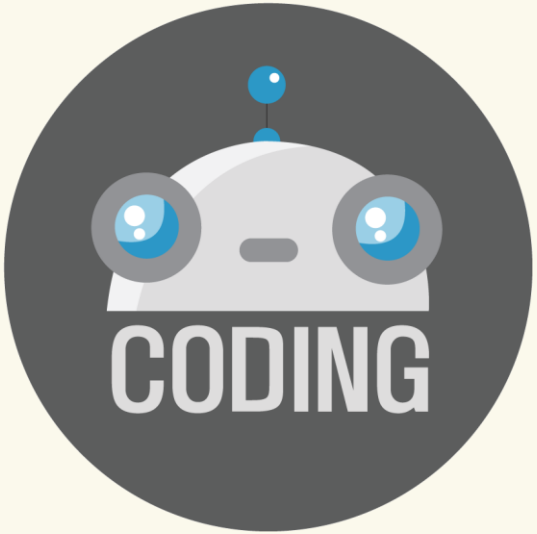
- TankController 스크립트의

전역변수인 Tank Color에 색상 지정

```
public class TankCotroller : MonoBehaviour
{
    public int playerNum = 1;        // 플레이어 번호
    string mvAxisName;                // 이동 축의 이름 저장
    string rotAxisName;               // 회전 축의 이름 저장
    float tkMvSpeed = 10.0f;        // 탱크의 이동 속도
    float tkRotSpeed = 150.0f;      // 탱크의 회전 속도
    public Color tankColor;

    void Start()
    {
        mvAxisName = "Vertical" + playerNum;
        rotAxisName = "Horizontal" + playerNum;

        // Get all of the renderers of the tank.
        MeshRenderer [ ] renderers = GetComponentsInChildren<MeshRenderer> ();

        // Go through all the renderers...
        for (int i = 0; i < renderers.Length; i++) {
            // ... set their material color to the color specific to this tank.
            renderers [ i ].material.color = tankColor;
        }
    }
    void Update()
    {
        float mv = Input.GetAxis (mvAxisName) * tkMvSpeed * Time.deltaTime;
        float rot = Input.GetAxis (rotAxisName) * tkRotSpeed * Time.deltaTime;

        transform.Translate (0 , 0 , mv);
        transform.Rotate (0 , rot , 0);
    }
}
```

2인용 게임 만들기





THANK YOU